

CLOUD & DEVOPS • CONTAINER ORCHESTRATION

Kubernetes

An Introduction to Container Orchestration

01 Core Concepts

02 Cluster Architecture

03 Networking Model

Vishwanath M S

Agenda

1

What is Kubernetes?

Origins, purpose, and what problem it actually solves

2

Kubernetes Primitives

Master & Node components, Pods, Replication Controllers, Services, Labels, Namespaces

3

Configuration & Cluster Setup Pointers

Config file differences, networking setup with Flannel overlay networking

4

Demo — Using MiniKube

Hands-on walkthrough of a local single-node cluster

What is Kubernetes?

Kubernetes is a container orchestration platform for applications that run on containers — a collaborative open-source project originally conceived by Google.



Born at Google

- Grew out of a decade of running billions of containers a week internally (the Borg system)
- Donated to the newly formed Cloud Native Computing Foundation (CNCF)
- Now the de-facto standard for orchestrating containerized workloads at scale

ALSO CALLED

Kube

K8s

That's 'k' + 8 letters + 's'

What Kubernetes Actually Does

Start, stop, update, and manage a cluster of machines running containers — in a consistent and maintainable way.



Best Suited For

Horizontally scalable, stateless, or 'microservices'-style application architectures — where you run many identical, replaceable copies of a workload.



Doesn't Exclude Others

Being best-suited for microservices does not mean other architectures won't work on Kubernetes, or that they're ignored by the platform.



Not a Docker CLI Clone

Kubernetes does NOT — and will not — expose all of the 'features' of the docker command line. It solves a different, cluster-level problem.

Kubernetes Key Words

Before diving into each one, here's the map: two physical building blocks, five virtual/logical ones.

PHYSICAL COMPONENTS

Master

The control-plane machine that runs the cluster brain

Minion / Node

A worker machine that actually runs your containers

VIRTUAL COMPONENTS

1

Pod

2

Replication Controller

3

Service

4

Label

5

Namespace

Master Node — Core Components

CONSISTS OF

kube-apiserver

Front door for all cluster operations — implements the API that every tool and library talks to

kube-scheduler

Allocates Pods to available nodes based on workload and resource utilization

kube-controller-manager

Non-terminating control loop that reconciles cluster state and reports back to the API server

etcd

Highly-available key/value store holding all configuration data, distributed across nodes

MAY ALSO CONTAIN

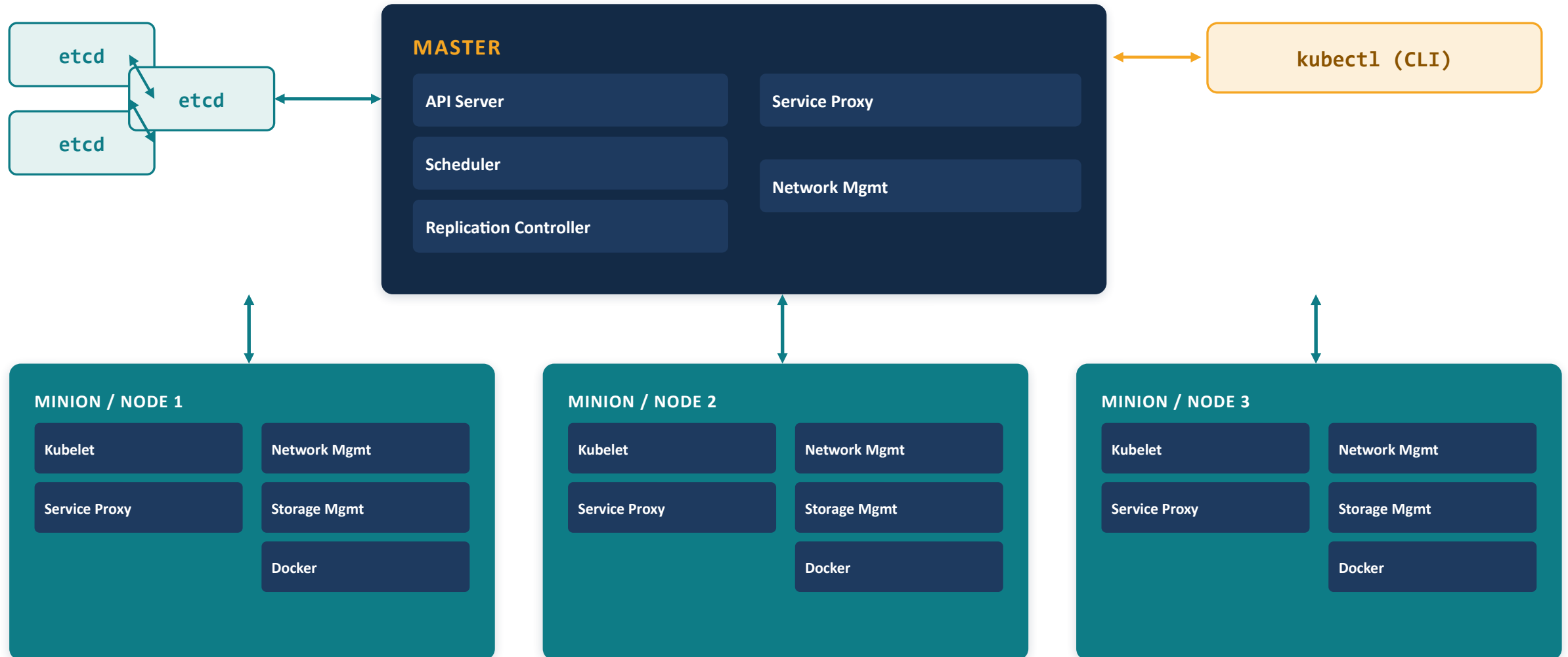
Kube-proxy

Network Mgmt Utility

WHY IT MATTERS

If the Master goes down, running Pods keep serving traffic — but nothing new can be scheduled, scaled, or reconciled until it's back.

Kubernetes Components



Minion / Node — Core Components

Every worker node in the cluster runs the same small set of agents:



kubelet

The primary node agent — registers the node, receives Pod specs from the API server, and ensures their containers are running and healthy.



kube-proxy

Maintains network rules on the node so Services can route traffic to the right backend Pods, wherever they're scheduled.



cAdvisor

Container Advisor — collects resource usage and performance metrics (CPU, memory, network, filesystem) from running containers.



Network Mgmt Utility

Handles the node's networking setup so Pods get connectivity consistent with the cluster's overlay network.

Pod — Key Properties

SCHEDULING & LIFECYCLE

- Single schedulable unit of work
- Can not move between machines
- Can not span machines
- One or more containers

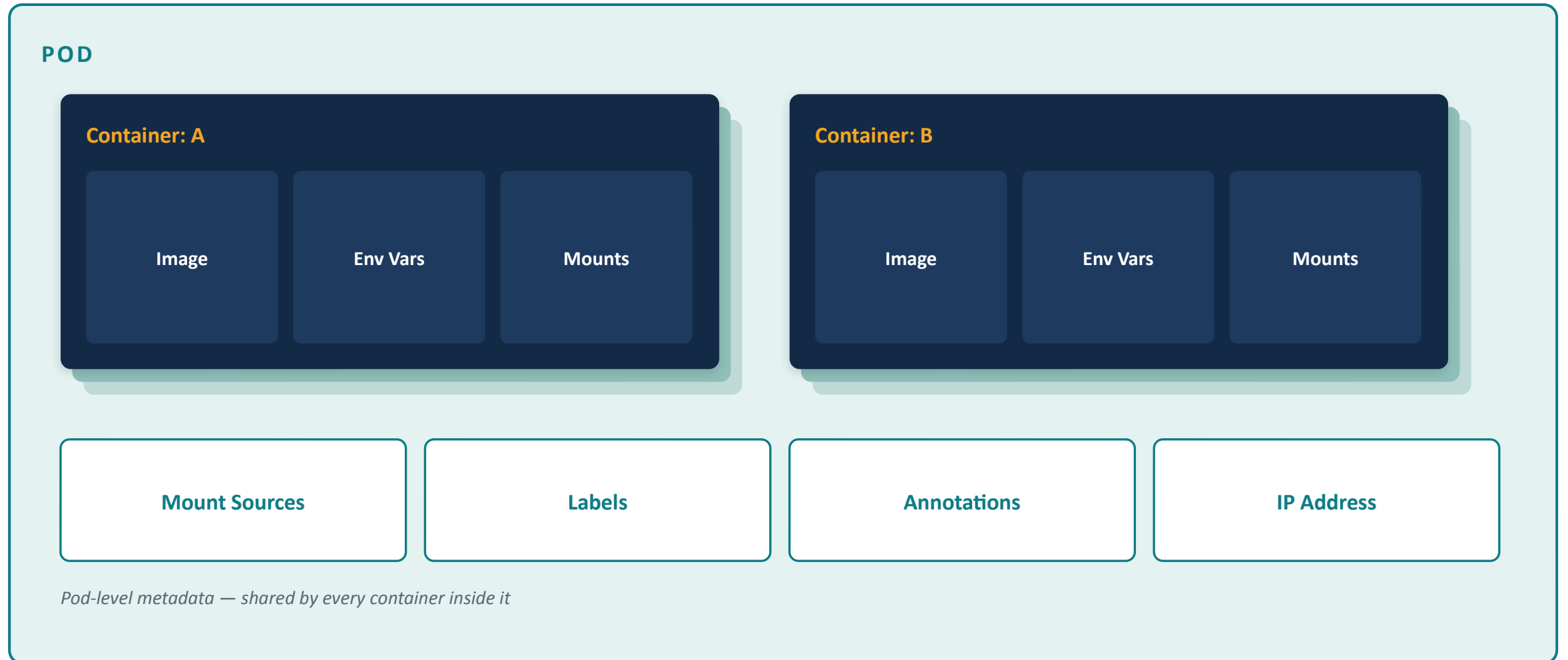
IDENTITY & CONFIGURATION

- Shared network namespace
- Metadata about the container(s)
- Env vars — configuration for the container
- Every Pod gets a unique IP



A Pod's IP is assigned by the container engine, not by Kube itself — a subtle but important distinction when debugging networking.

Anatomy of a Pod



Replication Controller

CONSISTS OF

1 Pod template

2 Count

3 Label Selector

HOW IT BEHAVES

Kube will try to keep \$count copies of Pods matching the label selector running, at all times.

If too few copies are running, the Replication Controller starts a new Pod somewhere else in the cluster — automatically, with no manual intervention.

This is the self-healing mechanism behind Kubernetes' reputation for resilience.

Services

How 'stuff' finds Pods, which could be running anywhere in the cluster.

A SERVICE ALWAYS DEFINES

- What port in the container to target
- Which Pod labels should respond to this type of request

A SERVICE CAN ALSO DEFINE

What the 'internal' IP should be

What the 'external' IP should be

What port the Service should listen on

Labels

A list of key=value pairs attached to (almost) every object in the cluster.

```
name: apache  
environment: qa  
version: v2
```

CURRENTLY USED IN 2 MAIN PLACES

1

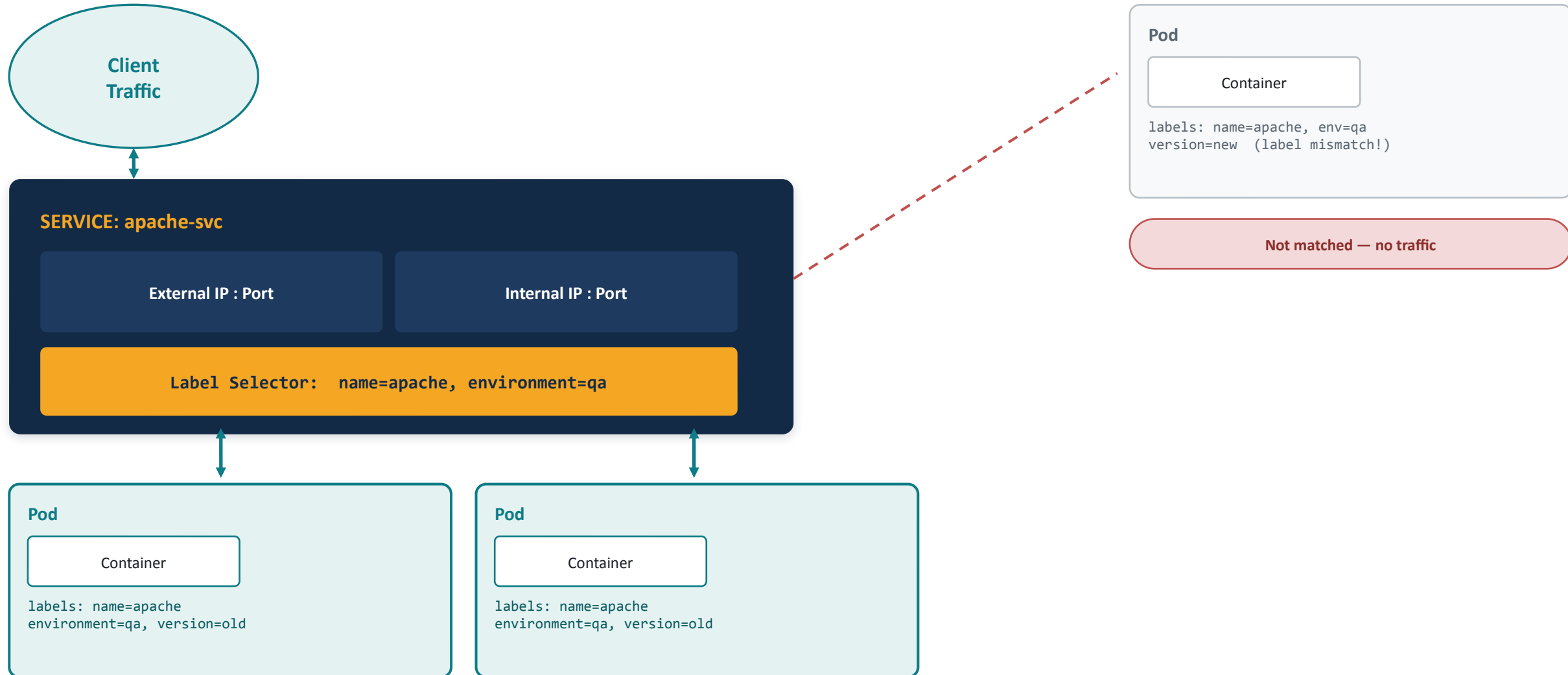
Matching Pods to Replication Controllers

2

Matching Pods to Services

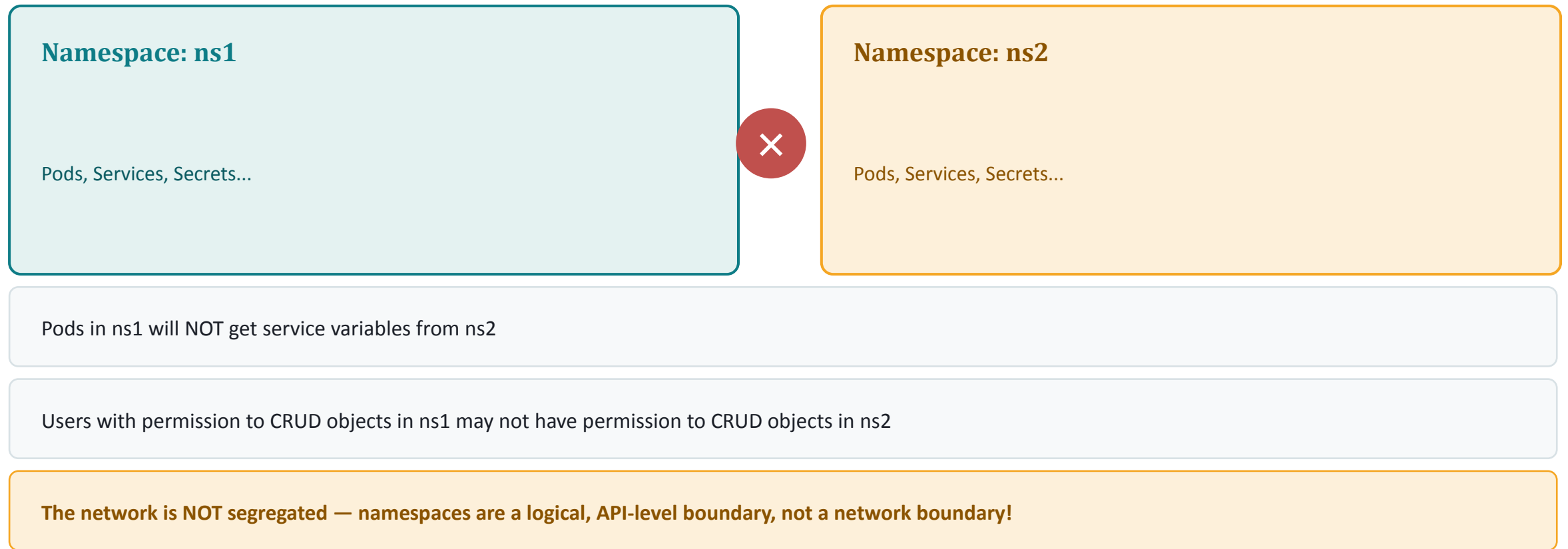
Objects can be queried from the API server by label — this is exactly what `kubectl get pods -l tier=backend` is doing.

Services & Labels — Rolling Update in Action



Namespaces

Attached to every object — a way to divide a cluster into isolated logical groups.



Configuration Differences

A few gotchas to expect when moving between Kubernetes versions or distributions.

CONFIGURATION CHANGES

Systemd unit files and `/etc/kubernetes/` file formatting vary across distributions and versions.

kubelet

```
--api_servers=<address>
```

Points the node agent at the control plane's API server

controller-manager

```
--machines=<list>
```

Historical flag for statically declaring the machine list (legacy setups)

Networking Setup



Networking is a problem in Docker — BUT NOT in Kube.

If any two Docker containers on any two hosts can talk to each other over IP, Kubernetes will just work. Kube makes those cross-host networking problems disappear for you.

WE USE THE FLANNEL FRAMEWORK FOR NETWORKING



Easy Configuration

Minimal setup to get every node speaking the same overlay network



VXLAN Overlay Network

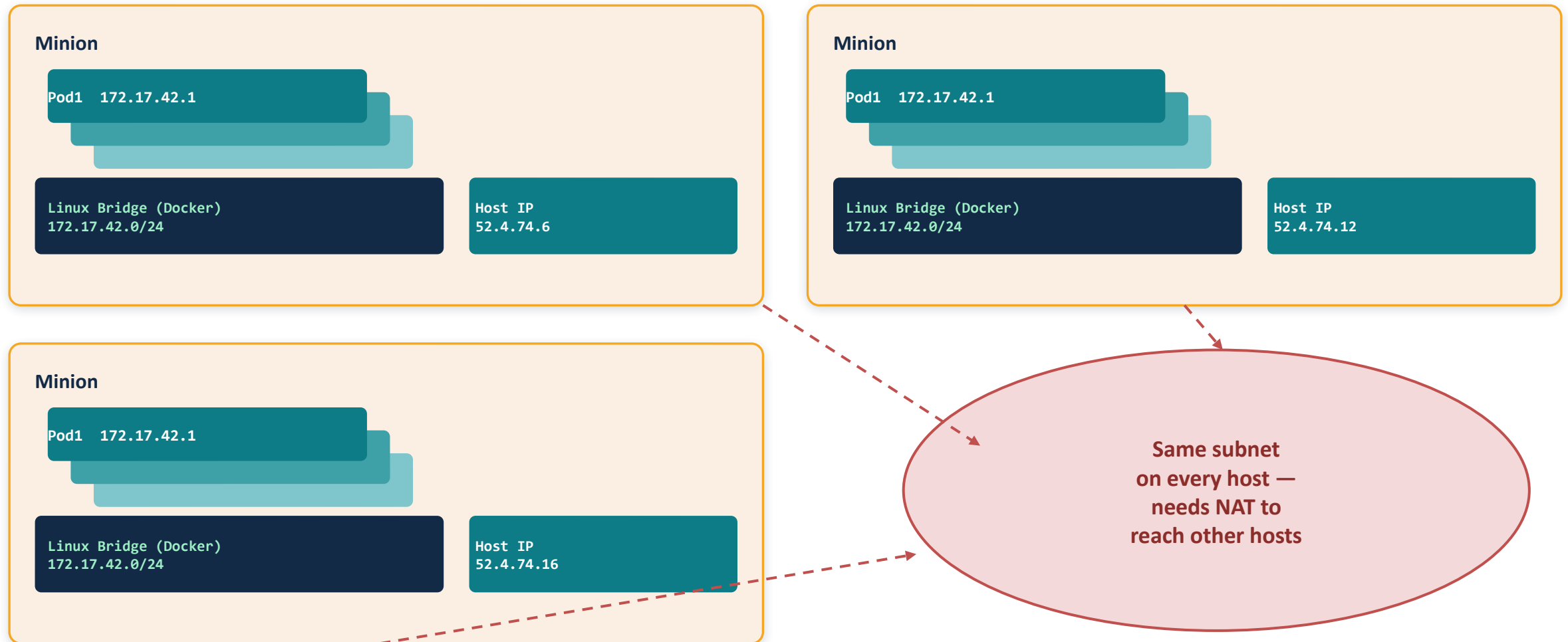
Creates a virtual network layer spanning all hosts in the cluster



Docker Integration

Configures Docker to launch Pods directly inside this overlay

Kube Networking Out of the Box



Custom Networking Setup

We use the Flannel Framework for networking:



Easy Configuration

A single config push gets every node participating in the overlay network.



VXLAN Overlay Network

Flannel creates a VXLAN overlay so each host gets its own unique Pod subnet.



Docker Launches Pods In This Overlay

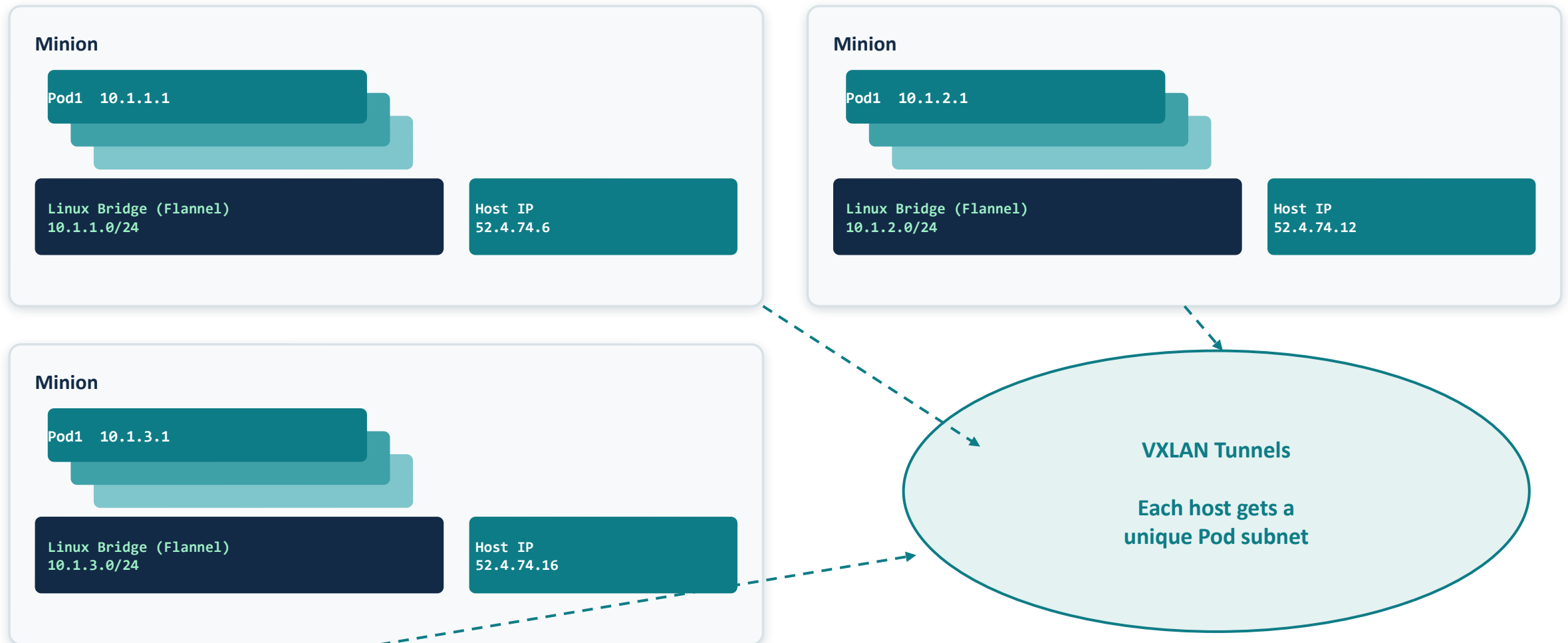
Docker on each node is configured to hand out IPs from that host's overlay subnet.



Inter-Communication of the Pods

Pods reach each other directly by IP across hosts — no manual NAT/port-mapping needed.

Networking With an Overlay Network



RECAP

You should now be able to...

- Explain what Kubernetes is and the class of problems it's built to solve
- Describe the Master / Minion (Node) architecture and their core components
- Define a Pod, a Replication Controller, a Service, a Label, and a Namespace
- Explain how Labels and Selectors connect Services to the right Pods
- Describe why Kubernetes needs an overlay network like Flannel, and what it fixes



Next: Live Demo — Using MiniKube